

课题11：面向 RAG 的智能分段与内容组织智能体实现规划

目录

1. 课题要求拆解
 2. 智能体定义
 3. 建设目标与能力边界
 4. 总体实现思路
 5. 系统总体架构
 6. 核心模块规划
 7. 输入输出与数据结构设计
 8. 分段策略设计
 9. 特殊内容保护方案
 10. 内容组织方案
 11. 原文锚点回链方案
 12. 闭环评测方案
 13. 进阶：QA / 指令数据合成
 14. 服务接口设计
 15. 数据库与文件组织设计
 16. 推荐技术栈
 17. 开发阶段规划
 18. 小组分工建议
 19. 验收指标对照
 20. 与开源/通用方案的差异
 21. 关键攻关点
 22. 风险与应对
 23. 最终交付物清单
 24. 附录：核心伪代码
-

1. 课题要求拆解

1.1 课题名称

面向 RAG 的智能分段与内容组织智能体

该课题属于任务书中的专项能力智能体，它不是完整 RAG 平台，也不是普通文本切分脚本，而是数据治理流水线中可被调用的一个独立能力服务。

在完整数据治理流程中，它位于口径归一之后、成果封装之前。也就是说，上游已经完成文档解析、清洗、脱敏、术语归一和实体标准化等工作，本智能体接收的是治理后的结构化全文，再把这些内容组织成适合 RAG 入库和后续问答的 chunk 序列。

1.2 任务书中的核心要求

根据任务书，智能体需要完成以下工作：

1. 输入治理后、口径已统一的结构化全文。
2. 按照结构/语义感知策略进行分段。
3. 分段时保证：
 - 不破句；
 - 表格整体成块；
 - 公式整体成块；
 - 代码整体成块；
 - chunk 长度可控；
 - 支持目标长度区间和片段重叠。
4. 为每个片段生成：
 - 文本内容；
 - 长度信息；
 - 主题标签；
 - 关键词标签；
 - 摘要；
 - 标准实体标签；
 - 原文锚点回链。
5. 使用下游检索指标闭环评估分段质量。
6. 与固定长度分段基线进行对比。
7. 进阶方向可探索：
 - 面向训练语料的高质量问答对生成；
 - 指令数据生成；
 - 可答性校验；

- 忠实度校验；
- 父子块/多粒度分段；
- 面向不同下游任务的策略切换。

1.3 期望输入

任务书给出的期望输入是：

治理后、口径已统一的结构化全文，含内容块与外置资产引用，带结构与锚点；分段策略与下游消费相关参数。

因此，本项目不应把主要精力放在从 PDF 或 Word 原文中抽取文本，而应假定上游已经提供了结构化表示。例如：

- 标题节点；
- 段落节点；
- 表格节点；
- 公式节点；
- 代码节点；
- 图片说明节点；
- 页码；
- 原文锚点；
- 来源文件信息；
- 已归一的实体和术语。

如果上游暂时无法提供完整结构化结果，可以在课程项目中设计一个简化适配器，把普通 Markdown、纯文本或 docx 抽取内容转换成统一节点格式。但这应作为适配层，而不是本课题的核心研究点。

1.4 期望输出

任务书给出的期望输出是：

语义完整的分段结果，每个片段含文本、长度、标签、摘要、实体标签、原文锚点回链；进阶可输出问答对/指令数据。

因此，输出不应只是字符串数组：

```
1 | ["第一段文本", "第二段文本"]
```

而应是结构化 chunk 序列:

```
1  {
2    "doc_id": "doc_001",
3    "chunks": [
4      {
5        "chunk_id": "doc_001_chunk_0001",
6        "title_path": ["一、系统概述", "1.1 建设背景"],
7        "content": "本片段正文内容...",
8        "chunk_type": "normal",
9        "token_count": 768,
10       "summary": "本片段说明系统建设背景和主要问题。",
11       "keywords": ["RAG", "智能分段", "知识库"],
12       "entity_tags": [
13         {
14           "name": "RAG",
15           "type": "technical_term",
16           "standard_id": "term_rag"
17         }
18       ],
19       "source_refs": [
20         {
21           "page_start": 3,
22           "page_end": 4,
23           "node_ids": ["n_12", "n_13", "n_14"]
24         }
25       ],
26       "strategy_info": {
27         "strategy": "structure_semantic_hybrid",
28         "split_reason": "二级标题边界 + 长度接近目标区间",
29         "merge_reason": null
30       }
31     ]
32   }
33 }
```

1.5 技术指标

指标项	建议目标
分段边界质量	不破句率 100%；表格/公式/代码整体成块率 $\geq 95\%$ ；目标长度区间命中率 $\geq 90\%$
下游检索提升	相对固定长度分段基线，RAG 检索 Recall@k / nDCG 提升 $\geq 10\%$ ，或给出显著提升证据
语义完整	片段内语义完整性人工评估 $\geq 85\%$
打标与摘要	内容标签准确率 $\geq 85\%$ ；摘要忠实度 $\geq 90\%$ ，无臆造
原文回链	片段到原文锚点回链完整率 100%
进阶 QA 合成	生成问答对可答性 $\geq 90\%$ ；忠实度 $\geq 90\%$

这些指标决定了项目实现时不能只做功能演示，还必须做质量验证。

1.6 交付要求

任务书要求最终交付：

1. 可运行的分段智能体一套；
2. 对外暴露标准接口；
3. 可供转换流水线调用；
4. 输入结构化全文；
5. 输出 chunk 序列；
6. 附与定长切分的下游检索对比实验；
7. 附示例数据。

因此，最终项目至少应包含：

- 后端服务；
- 核心分段算法；
- 内容组织模块；
- 评测模块；
- 示例数据；
- 接口文档；
- 运行说明；
- 实验结果。

2. 智能体定义

2.1 本课题中的智能体是什么

本课题中的智能体不是聊天机器人，也不是完全开放式自主 Agent。它更接近一个有明确任务目标、明确输入输出、明确流程约束、可被其他系统调用的专项能力服务。

可以定义为：

面向 RAG 的智能分段与内容组织智能体，是一个服务于数据治理流水线和知识库入库流程的专项能力智能体。它接收治理后、口径已统一的结构化长文档，自动分析标题层级、段落语义、表格、公式、代码和来源锚点，按照结构/语义感知策略生成语义完整、长度可控、来源可追溯的 chunk 序列，并为每个 chunk 生成标签、摘要、实体标签和原文回链，同时通过下游 RAG 检索指标评估分段质量。

2.2 智能体的“智能”体现在哪里

本课题的智能性主要体现在以下方面：

1. 不是固定长度机械切分
系统需要根据标题、段落、语义相似度和内容类型判断边界。
2. 能根据内容类型选择策略
普通段落、表格、公式、代码、列表、条款使用不同处理方式。
3. 能自动处理过长和过短片段
过长片段需要继续拆分，过短片段需要合并或补充上下文。
4. 能自动补充组织信息
为 chunk 生成标题路径、摘要、关键词、实体标签和来源锚点。
5. 能用评测结果反向验证策略
不只看分段是否整齐，还要看下游检索是否更准。
6. 可作为服务被其他智能体调用
例如数据格式标准化转换智能体可以调用本智能体完成分段和内容组织。

2.3 智能体与普通程序的区别

普通文本切分脚本通常是：

```
1 | 输入一段文本
2 | 每 800 字切一次
3 | 输出若干文本段
```

本智能体的流程是：

- 1 输入结构化长文档
- 2 识别标题、段落、表格、公式、代码和锚点
- 3 根据结构和语义生成候选分段
- 4 检查长度、句子边界和特殊块完整性
- 5 对过长片段拆分，对过短片段合并
- 6 生成摘要、关键词、实体标签和来源回链
- 7 输出标准化 `chunk` 序列
- 8 运行检索评测并与基线比较

因此，它不是一个单一函数，而是一个组合多种能力的工程化智能处理单元。

3. 建设目标与能力边界

3.1 总体目标

建设一个可独立运行、可被流水线调用、可评测、可复现的智能分段与内容组织智能体。它应能将结构化长文档加工为高质量 `chunk`，使下游 RAG 系统更容易检索到完整、准确、可解释的信息。

3.2 具体目标

1. 结构感知
利用标题层级、段落、列表、表格、公式、代码块等结构信息进行分段。
2. 语义感知
利用语义相似度、主题变化和段落连贯性优化分段边界。
3. 长度可控
支持最小长度、目标长度、最大长度和 `overlap` 参数。
4. 特殊块保护
表格、公式、代码整体成块，不在内部随意截断。
5. 内容组织
为 `chunk` 生成摘要、关键词、主题标签、实体标签和管理标签。
6. 原文回链
每个 `chunk` 能追溯到原文页码、节点、字符范围或锚点。
7. 闭环评测
使用 RAG 检索指标评价分段效果，并与固定长度基线对比。
8. 服务化集成
通过 HTTP API 或等价方式对外提供能力。

3.3 本智能体做什么

本智能体负责：

- 接收结构化全文；
- 解析文档节点树；
- 生成候选 chunk；
- 按结构、语义和长度优化边界；
- 保护表格、公式、代码；
- 为 chunk 添加摘要和标签；
- 建立原文锚点回链；
- 输出 chunk 序列；
- 统计分段质量；
- 执行检索评测；
- 输出评测报告。

3.4 本智能体不做什么

为了控制项目边界，本智能体不重点做：

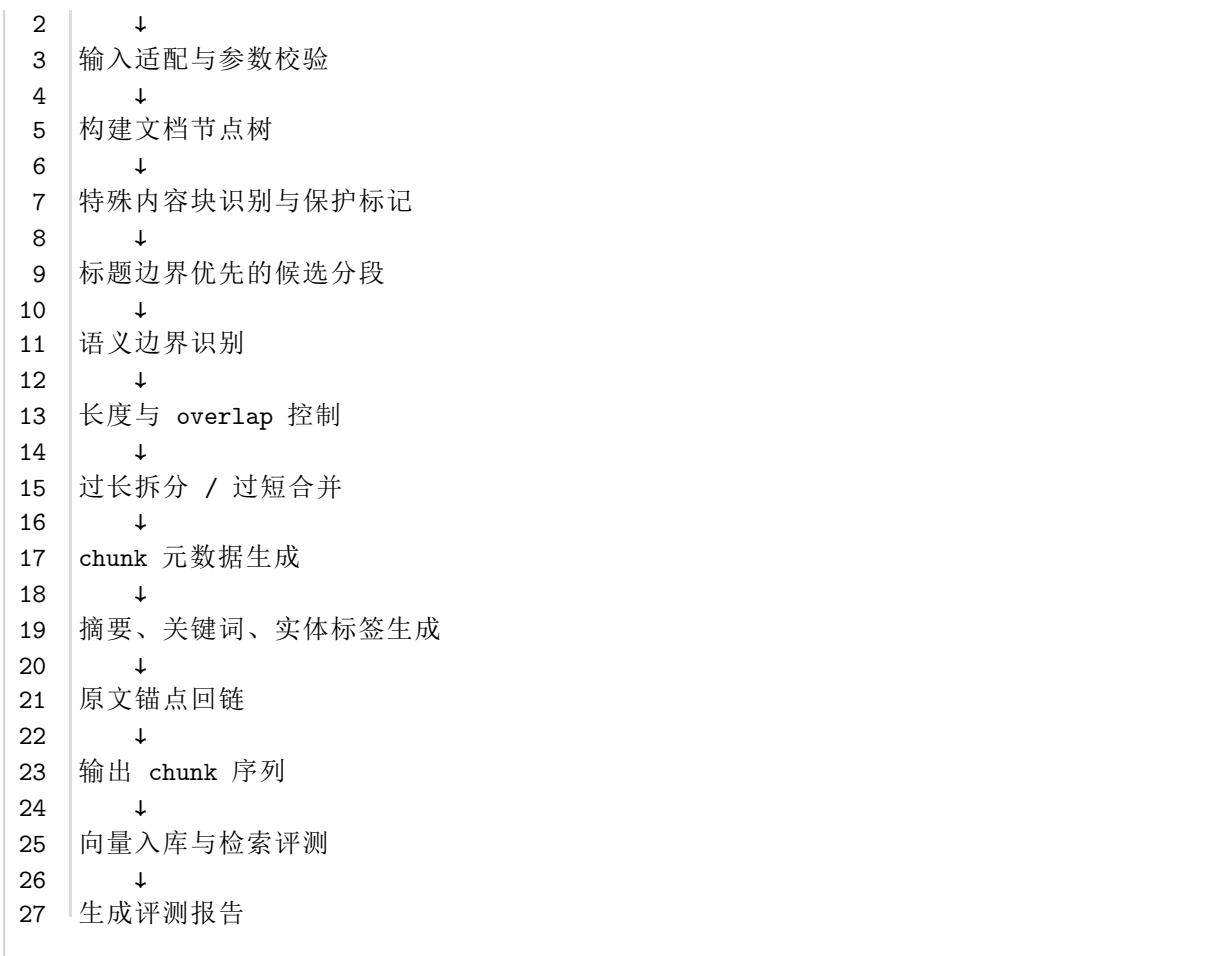
1. 不负责完整 OCR
OCR 和复杂版面解析应由上游解析智能体或现有解析工具完成。
2. 不负责完整 RAG 平台
本智能体只负责入库前分段与内容组织，不负责完整问答产品。
3. 不负责用户权限和知识库管理系统
不实现多租户、权限、审计后台等平台功能。
4. 不负责行业知识库全面构建
可以使用已有词表或小型实体表，不要求建设完整行业知识图谱。
5. 不负责大规模生产级部署
课程项目以可运行、可演示、可评测为主。

4. 总体实现思路

4.1 总体流程

推荐实现流程如下：

1	结构化全文输入
---	---------



4.2 实现原则

- 1. 规则优先，模型辅助
标题识别、长度控制、表格保护、回链等尽量使用确定性规则；语义边界、摘要和标签可使用模型。
- 2. 结构优先，语义补充
分段优先尊重标题和自然段结构，再用语义相似度优化边界。
- 3. 不破坏内容块
表格、公式、代码、编号条款默认不从中间截断。
- 4. 结果可解释
每个 chunk 应记录切分原因、合并原因和来源节点。
- 5. 评测可复现
同一输入、同一配置、同一模型版本应得到可复现结果。
- 6. 服务化接口稳定
输入输出使用明确 schema，便于与上下游解耦。

4.3 最小可用版本与增强版本

为了保证项目能落地，建议分为三个版本：

版本	能力范围	目标
V1 基础版	标题优先分段、长度控制、来源回链	跑通完整流程
V2 增强版	语义边界、过长拆分、过短合并、特殊块保护、摘要标签	达到任务书主要要求
V3 评测版	RAG 检索评测、多策略对比、QA 合成可选	形成实验闭环和最终报告

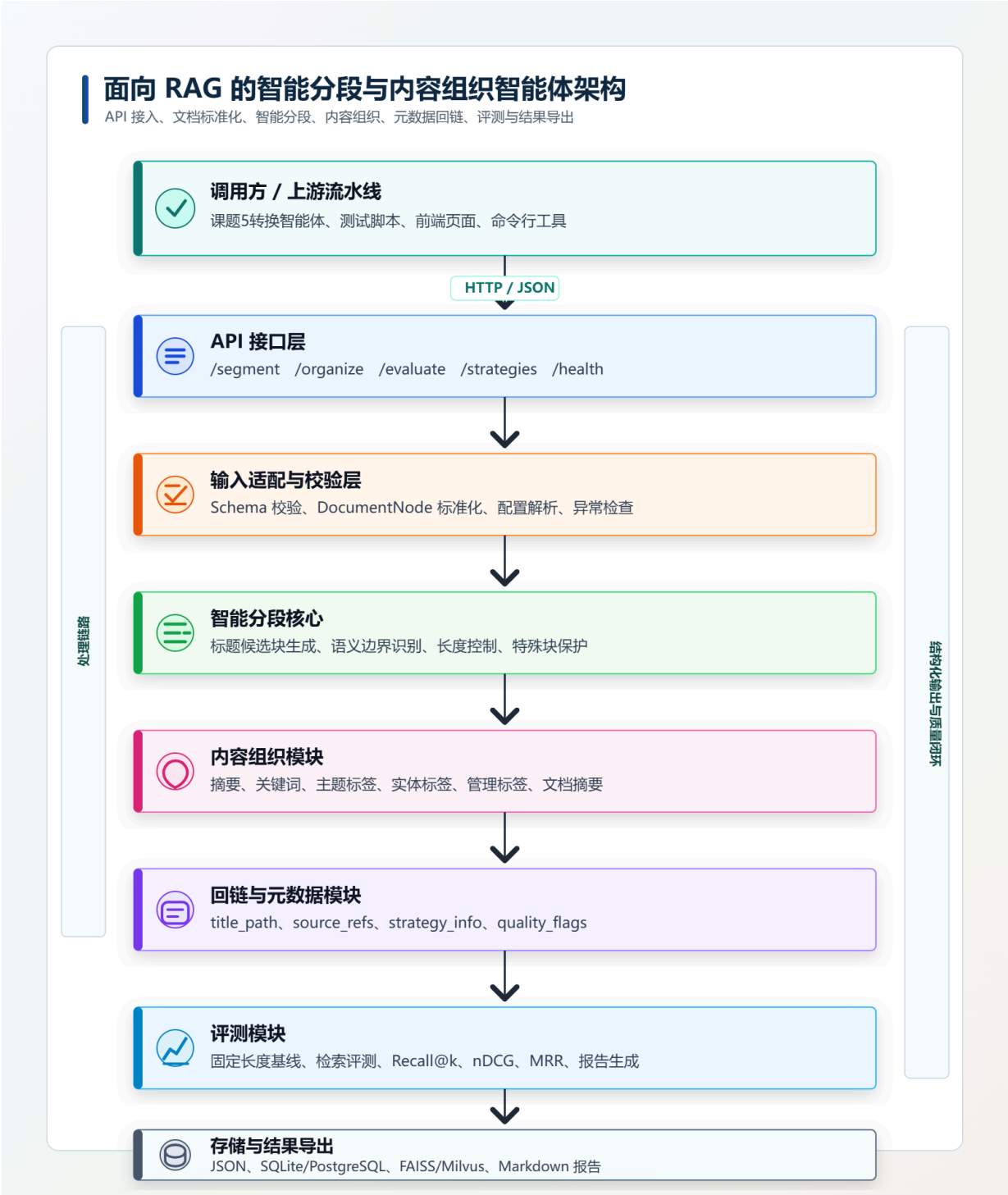
5. 系统总体架构

5.1 架构分层

系统建议分为 7 层：

- 1. 接口层
提供 HTTP API，接收文档与配置，返回 chunk 结果和评测结果。
- 2. 输入适配层
将不同输入格式转换为统一 DocumentNode 结构。
- 3. 分段决策层
负责候选分段、语义边界、长度控制、特殊块保护。
- 4. 内容组织层
负责摘要、关键词、实体标签、管理标签、文档级摘要。
- 5. 回链与元数据层
负责 source_refs、title_path、chunk_id、strategy_info 等生成。
- 6. 评测层
负责基线分段、向量入库、检索、指标计算和报告生成。
- 7. 存储与日志层
存储任务记录、配置、chunk、评测结果和处理日志。

5.2 架构图



5.3 运行模式

建议支持三种运行模式：

1. 服务模式
启动 FastAPI 服务，供上游智能体调用。
2. 命令行模式
本地读取示例文件，输出 chunk JSON 和评测报告。

3. 演示模式

通过轻量前端页面查看文档结构、chunk、标签、摘要和检索结果。

6. 核心模块规划

6.1 API 服务模块

职责：

- 接收请求；
- 校验输入；
- 调用分段流程；
- 返回结果；
- 记录任务日志；
- 支持健康检查。

主要接口：

- `POST /v1/segment`
- `POST /v1/evaluate`
- `POST /v1/organize`
- `GET /v1/strategies`
- `GET /v1/jobs/{job_id}`
- `GET /health`

6.2 输入适配模块

职责：

- 接收结构化全文；
- 校验 `doc_id`、`nodes`、`metadata`、`config`；
- 将上游不同字段转换为统一 `DocumentNode`；
- 补全缺失字段；
- 识别异常节点；
- 保留原始输入快照。

输入适配模块需要处理以下情况：

- 标题层级缺失；
- 段落为空；

- 表格没有表名；
- 代码块语言未知；
- 页码缺失；
- 节点顺序不稳定；
- 来源锚点不完整。

处理原则：

1. 能修正则修正；
2. 不能修正则降级；
3. 降级也要保留原始节点；
4. 所有异常写入 `warnings`。

6.3 文档结构树模块

职责：

- 根据 `heading` 节点构建标题树；
- 为每个节点补充 `title_path`；
- 建立父子关系；
- 建立节点顺序；
- 建立页码和锚点索引。

示例：

```
1 | 一级标题：系统建设背景
2 |   二级标题：业务背景
3 |     段落 1
4 |     段落 2
5 |   二级标题：技术背景
6 |     表格 1
7 |     段落 3
```

每个段落、表格、公式、代码都应能找到它所属的标题路径。

6.4 特殊块识别模块

职责：

- 识别表格节点；
- 识别公式节点；
- 识别代码节点；
- 识别列表节点；

- 识别图注和说明；
- 标记是否可拆分；
- 标记是否需要上下文补全。

特殊块默认策略：

类型	默认策略
table	整体成块，不与无关段落混合
formula	与解释文字优先合并，不单独孤立
code	与代码说明、函数注释优先合并
list	尽量保留连续列表项
heading	作为上下文，不单独形成内容 chunk

6.5 候选分段模块

职责：

- 按标题层级生成初始候选块；
- 按自然段落组合内容；
- 保留特殊块；
- 生成候选 chunk 列表。

基本逻辑：

1. 一级标题下内容过长时，按二级标题拆；
2. 二级标题下内容过长时，按自然段拆；
3. 自然段仍过长时，按句子边界拆；
4. 表格、公式、代码先保持整体；
5. 候选块必须保留标题路径。

6.6 语义边界识别模块

职责：

- 计算段落或句子的 embedding；
- 计算相邻语义相似度；
- 找到主题变化点；
- 为候选边界打分。

建议使用：

- 中文场景：`bge-small-zh`、`bge-base-zh`、`text2vec`；

- 中英文混合： `bge-m3` ；
- 资源有限：先使用 TF-IDF 或关键词重叠作为轻量替代。

语义边界不是唯一依据，它只作为边界评分的一部分。

6.7 长度与重叠控制模块

职责：

- 统计 chunk token 数；
- 控制最小长度；
- 控制目标长度；
- 控制最大长度；
- 处理 overlap；
- 对过长块拆分；
- 对过短块合并。

推荐参数：

```
1 min_tokens: 256
2 target_tokens: 768
3 max_tokens: 1024
4 overlap_tokens: 80
5 hard_max_tokens: 1400
```

说明：

- `target_tokens` 是理想长度；
- `max_tokens` 是常规上限；
- `hard_max_tokens` 是特殊块允许的最高上限；
- 表格或代码块可适当突破 `max_tokens`，但不应突破 `hard_max_tokens`。

6.8 分段后处理模块

职责：

- 为 chunk 生成稳定 ID；
- 连接前后 chunk；
- 补充标题路径；
- 补充来源节点；
- 生成质量标记；
- 去除重复 overlap；

- 检查空 chunk；
- 检查超长 chunk。

质量标记示例：

```
1 {
2   "quality_flags": [
3     "contains_table",
4     "length_in_target_range",
5     "source_refs_complete"
6   ]
7 }
```

异常标记示例：

```
1 {
2   "quality_flags": [
3     "too_long",
4     "missing_page_ref",
5     "summary_not_generated"
6   ]
7 }
```

6.9 内容打标模块

职责：

- 生成主题标签；
- 生成关键词；
- 生成管理标签；
- 生成质量标签；
- 对标签去重；
- 对标签规范化。

标签类型：

标签类型	说明	推荐实现
内容标签	主题、关键词、技术词	规则 + embedding / LLM
管理标签	文档类型、密级、版本、来源	规则
质量标签	过长、缺页码、含表格、含代码	规则
实体标签	标准实体、术语、机构、人名	规则 + 实体链接结果

6.10 摘要生成模块

职责：

- 生成 chunk 级摘要；
- 生成章节级摘要；
- 生成文档级摘要；
- 检查摘要是否忠实；
- 标记摘要生成方式。

摘要生成原则：

1. 只基于原文；
2. 不补充外部信息；
3. 不引入推测性结论；
4. 保留关键事实；
5. 控制长度；
6. 对表格或代码摘要应说明其用途和主要字段。

6.11 实体标签与回链模块

职责：

- 写入标准实体标签；
- 保留实体来源；
- 建立 chunk 到原文节点的映射；
- 建立 chunk 到页码的映射；
- 支持前端跳转展示。

如果上游已经完成实体链接，本模块直接读取上游实体结果。如果上游没有实体结果，可以先实现轻量规则版，例如识别：

- 英文缩写；
- 专业术语；
- 机构名；
- 产品名；
- 指标名；
- 技术组件名。

6.12 评测模块

职责：

- 生成固定长度切分基线；
- 生成标题切分基线；
- 生成智能分段结果；
- 构建向量索引；
- 执行检索；
- 计算指标；
- 导出评测报告。

核心指标：

- Recall@k；
 - nDCG@k；
 - MRR；
 - 首个相关 chunk 排名；
 - 命中率；
 - 特殊块保护率；
 - 长度命中率；
 - 摘要忠实度；
 - 原文回链完整率。
-

7. 输入输出与数据结构设计

7.1 输入数据结构

样例输入：

```
1  {
2    "doc_id": "doc_001",
3    "doc_title": "示例技术文档",
4    "doc_type": "technical_report",
5    "metadata": {
6      "source_file": "example.docx",
7      "language": "zh",
8      "version": "v1",
9      "normalized": true
10   },
11   "nodes": [],
```

```
12   "assets": [],
13   "config": {}
14 }
```

7.2 DocumentNode 结构

```
1  {
2    "node_id": "n_0001",
3    "node_type": "paragraph",
4    "level": null,
5    "text": "本段正文内容。",
6    "parent_id": "h_0001",
7    "order": 12,
8    "page_start": 3,
9    "page_end": 3,
10   "anchor": {
11     "source_file": "example.docx",
12     "page": 3,
13     "block_id": "block_12",
14     "char_start": 1200,
15     "char_end": 1360
16   },
17   "metadata": {
18     "font_size": 12,
19     "is_list_item": false,
20     "contains_entity": true
21   }
22 }
```

7.3 节点类型枚举

node_type	说明
heading	标题
paragraph	普通段落
list_item	列表项
table	表格
formula	公式
code	代码块
quote	引用
image_caption	图片说明
footnote	脚注
appendix	附录

7.4 分段配置结构

```
1  {
2    "strategy": "structure_semantic_hybrid",
3    "min_tokens": 256,
4    "target_tokens": 768,
5    "max_tokens": 1024,
6    "hard_max_tokens": 1400,
7    "overlap_tokens": 80,
8    "preserve_table": true,
9    "preserve_formula": true,
10   "preserve_code": true,
11   "semantic_split": true,
12   "semantic_threshold": 0.62,
13   "title_path_injection": true,
14   "generate_summary": true,
15   "generate_keywords": true,
16   "generate_entity_tags": true,
17   "return_debug": true
18 }
```

7.5 Chunk 输出结构

```
1  {
2    "chunk_id": "doc_001_chunk_0001",
3    "doc_id": "doc_001",
4    "chunk_index": 1,
5    "chunk_type": "normal",
6    "title_path": ["一、背景", "1.1 课题背景"],
7    "content": "chunk 正文内容。",
8    "content_with_context": "一、背景 > 1.1 课题背景\nchunk 正文内容。",
9    "summary": "本片段介绍课题背景。",
10   "keywords": ["RAG", "分段", "知识库"],
11   "topic_tags": ["智能分段", "数据治理"],
12   "management_tags": ["doc_type:technical_report"],
13   "entity_tags": [],
14   "token_count": 732,
15   "char_count": 1050,
16   "source_refs": [],
17   "prev_chunk_id": null,
18   "next_chunk_id": "doc_001_chunk_0002",
19   "strategy_info": {},
20   "quality_flags": []
21 }
```

7.6 source_refs 结构

```
1  {
2    "source_file": "example.docx",
3    "page_start": 3,
4    "page_end": 4,
5    "node_ids": ["n_0012", "n_0013"],
6    "block_ids": ["block_12", "block_13"],
7    "char_start": 1200,
8    "char_end": 1800,
9    "anchor_type": "page_block"
10 }
```

7.7 strategy_info 结构

```
1 {
2   "strategy": "structure_semantic_hybrid",
3   "candidate_source": "heading_section",
4   "split_reason": "语义边界 + 接近最大长度",
5   "merge_reason": null,
6   "semantic_score_before": 0.82,
7   "semantic_score_after": 0.43,
8   "length_policy": "target_range",
9   "special_block_policy": "preserve_table",
10  "overlap_applied": true
11 }
```

7.8 statistics 输出结构

```
1 {
2   "doc_id": "doc_001",
3   "chunk_count": 42,
4   "avg_tokens": 735,
5   "min_tokens": 240,
6   "max_tokens": 1180,
7   "target_range_hit_rate": 0.91,
8   "sentence_break_rate": 0.0,
9   "table_preserve_rate": 1.0,
10  "formula_preserve_rate": 1.0,
11  "code_preserve_rate": 1.0,
12  "source_ref_complete_rate": 1.0,
13  "warnings": []
14 }
```

8. 分段策略设计

8.1 为什么不能只用固定长度

固定长度切分的典型问题：

1. 把完整句子切断；
2. 把表格和表头分开；
3. 把代码和说明分开；
4. 把标题和正文分开；

5. 把同一论点拆成多个片段；
6. 把多个主题混在一个长片段里；
7. 检索时召回片段缺少上下文；
8. 回答时引用来源不清晰。

因此，本课题必须采用结构和语义结合的策略。

8.2 分段策略总体设计

推荐采用四级策略：

- 1 第一级：标题边界优先
- 2 第二级：自然段与句子边界
- 3 第三级：语义相似度边界
- 4 第四级：长度与特殊块约束

8.3 标题边界优先

标题是文档结构最重要的线索。分段时应优先保留标题下的内容完整性。

基本规则：

1. 一级标题通常不直接形成 `chunk`；
2. 二级或三级标题下内容长度合适时，可形成一个 `chunk`；
3. 标题下内容过长时，在其内部继续拆分；
4. 标题下内容过短时，可与同一父标题下相邻内容合并；
5. 每个 `chunk` 都保留完整 `title_path`。

8.4 自然段边界

自然段是最基本的语义单元。系统应尽量在自然段之间切分，而不是段落内部。

段落内部拆分只在以下情况出现：

- 段落过长；
- 段落包含多个明显主题；
- 段落中存在编号列表；
- 已接近最大 `token` 限制。

8.5 句子边界

任务书要求不破句，因此句子边界是硬约束。

中文句子切分可基于：

-
-
-
-
- 换行；
- 编号列表标识。

需要注意：

- 小数点不能误判；
- 英文缩写不能误判；
- 公式中的点号不能误判；
- URL 不能误判。

8.6 语义边界

语义边界用于识别主题变化。例如：

- | | |
|---|------------|
| 1 | 上一段讲系统背景 |
| 2 | 下一段开始讲技术架构 |

若两段 embedding 相似度明显下降，则该位置可能适合作为切分点。

推荐计算：

```
1 | similarity(i, i+1) = cosine(embedding(paragraph_i), embedding(paragraph_i+1))
```

如果相似度低于阈值，且当前累计长度接近目标区间，则优先切分。

8.7 边界评分公式

可设计一个综合边界评分：


```
1 score(boundary) =
2   w_title * title_boundary_score
3   + w_sentence * sentence_boundary_score
4   + w_semantic * semantic_change_score
5   + w_length * length_score
6   + w_special * special_block_score
7   - w_break * break_penalty
```

各项含义：

项	含义
title_boundary_score	是否在标题或小节边界
sentence_boundary_score	是否在句子边界
semantic_change_score	是否存在语义突变
length_score	当前长度是否接近目标区间
special_block_score	是否避开表格、公式、代码内部
break_penalty	是否破坏句子或特殊块

8.8 过长拆分策略

当候选 chunk 超过 `max_tokens` 时：

- 1. 优先按子标题拆；
- 2. 其次按自然段拆；
- 3. 再按句子拆；
- 4. 最后按 token 硬切，但必须避免破句；
- 5. 特殊块超过上限时，标记 `oversized_special_block`。

8.9 过短合并策略

当 chunk 小于 `min_tokens` 时：

- 1. 优先与同一标题路径下的后一个 chunk 合并；
- 2. 若后一个不合适，则与前一个合并；
- 3. 合并前检查是否超过 `max_tokens`；
- 4. 合并时保留完整 `source_refs`；
- 5. 合并原因写入 `merge_reason`。

8.10 overlap 策略

overlap 不应机械复制固定字数，而应按上下文需求生成。
推荐策略：

场景	overlap 内容
普通段落	上一 chunk 末尾 1 至 2 句
标题切换	标题路径
表格	表名 + 表头
代码	函数名 + 说明
条款	上级条款编号

这样能减少冗余，同时保留必要上下文。

9. 特殊内容保护方案

9.1 表格保护

表格是 RAG 中最容易被错误切分的内容之一。
表格保护策略：

- 1. 短表格整体成块；
- 2. 表格前的说明文字优先合并；
- 3. 表格标题必须保留；
- 4. 表头必须保留；
- 5. 长表格可按行组拆分；
- 6. 拆分长表格时每个子 chunk 重复表头；
- 7. 表格 source_refs 必须完整。

9.2 表格转文本格式

表格可同时保存两种形态：

- 1. 原始结构化表格；
- 2. 适合 embedding 的文本表述。

示例：

```
1 表名：系统模块说明
2
3 | 模块 | 功能 | 输出 |
4 |---|---|---|
5 | 解析模块 | 识别标题和段落 | 文档节点 |
6 | 分段模块 | 生成 chunk | chunk 序列 |
```

转为文本时可生成：

```
1 表格"系统模块说明"包含模块、功能、输出三列。
2 解析模块的功能是识别标题和段落，输出文档节点。
3 分段模块的功能是生成 chunk，输出 chunk 序列。
```

9.3 公式保护

公式不能单独孤立，否则检索出来后难以理解。

策略：

1. 公式与前文解释优先合并；
2. 公式与后文变量说明优先合并；
3. 公式节点保留 LaTeX 或文本形式；
4. 公式 `source_refs` 单独记录；
5. 摘要中说明公式用途。

9.4 代码保护

代码块处理策略：

1. 代码块整体保留；
2. 保留语言类型；
3. 保留函数名或类名；
4. 保留注释；
5. 与代码说明文字合并；
6. 不在函数中间截断；
7. 过长代码块可按函数拆分。

9.5 列表和编号条款保护

列表项通常语义连续，不应随机拆开。

策略：

1. 连续列表项尽量同块；
 2. 条款编号保留；
 3. 上级条款标题补充到 chunk；
 4. 拆分时保持编号连续；
 5. 列表过长时按一级列表项拆分。
-

10. 内容组织方案

10.1 内容组织目标

内容组织的目标是让每个 chunk 成为可检索、可理解、可追溯的知识单元。

因此，每个 chunk 不只是正文，还应包含：

- 标题路径；
- 摘要；
- 关键词；
- 主题标签；
- 管理标签；
- 实体标签；
- 来源页码；
- 来源节点；
- 切分策略信息；
- 质量标记。

10.2 关键词生成

关键词生成可采用三种方式：

1. 规则抽取；
2. TF-IDF / TextRank；
3. LLM 生成。

推荐实现：

- 1 第一步：规则抽取专业词、英文缩写、编号术语
- 2 第二步：使用 **TF-IDF** 或 **TextRank** 补充关键词
- 3 第三步：可选使用 **LLM** 对关键词去重和规范化

10.3 主题标签生成

主题标签应比关键词更高层。例如：

- 1 关键词：Recall@k、nDCG、向量检索
- 2 主题标签：RAG 评测

主题标签可通过：

- 预设标签体系；
- 标题路径映射；
- embedding 聚类；
- LLM 分类。

课程项目中建议先建立小型标签体系：

- 1 RAG：
 - 2 - 文档分段
 - 3 - 向量检索
 - 4 - 内容组织
 - 5 - 评测指标
- 6 数据治理：
 - 7 - 结构化
 - 8 - 清洗
 - 9 - 归一
 - 10 - 可追溯

10.4 管理标签

管理标签由规则生成，不建议使用大模型。

示例：

- `doc_type:technical_report`
- `source:course_project`
- `contains_table:true`
- `contains_code:false`
- `quality:source_ref_complete`
- `strategy:structure_semantic_hybrid`

10.5 摘要生成

摘要生成分为两级：

1. chunk 级摘要；
2. 文档级摘要。

chunk 级摘要要求：

- 50 到 150 字；
- 只概括本 chunk；
- 不引入外部信息；
- 不改写关键数值；
- 表格摘要应说明表格主题和核心字段。

文档级摘要可以先汇总 chunk 摘要，再生成整体摘要。

10.6 摘要忠实度控制

摘要忠实度控制方法：

1. 提示词明确要求“不得加入原文没有的信息”；
2. 摘要后进行关键词覆盖检查；
3. 抽样人工检查；
4. 对低置信摘要标记 `summary_low_confidence`；
5. 重要数值不允许模型改写；
6. 表格摘要尽量使用原始字段名。

10.7 实体标签写入

实体标签来源：

1. 上游实体链接结果；
2. 规则识别；
3. 小型词表；
4. LLM 辅助识别。

实体标签结构：

```
1 {
2   "mention": "RAG",
3   "standard_name": "Retrieval-Augmented Generation",
4   "entity_type": "technical_term",
5   "standard_id": "term_0001",
6   "source": "term_dict",
7   "confidence": 0.98
8 }
```

11. 原文锚点回链方案

11.1 回链的重要性

RAG 系统的可信度取决于能否说明答案来自哪里。
如果 `chunk` 没有来源信息，会出现以下问题：

- 检索结果无法验证；
- 回答引用不可信；
- 错误难以追踪；
- 无法回滚；
- 无法审计。

11.2 回链粒度

建议支持多级回链：

粒度	说明
文件级	来源文件名、版本、哈希
页级	page_start、page_end
块级	node_id、block_id
字符级	char_start、char_end
表格级	table_id、row_range、col_range

11.3 回链生成方式

生成 chunk 时，将合并进 chunk 的所有节点 source_refs 取并集。
例如：

```
1  "source_refs": [  
2    {  
3      "node_id": "n_12",  
4      "page_start": 3,  
5      "page_end": 3  
6    },  
7    {  
8      "node_id": "n_13",  
9      "page_start": 3,  
10     "page_end": 4  
11   }  
12 ]
```

11.4 回链完整性检查

检查规则：

1. 每个 chunk 至少有一个 source_ref;
2. source_ref 中必须有 node_id;
3. 如果上游提供页码，则必须保留页码;
4. 表格 chunk 必须保留 table_id;
5. 合并 chunk 必须保留所有来源节点。

目标：

```
1 | 片段 → 原文锚点回链完整率 = 100%
```

12. 闭环评测方案

12.1 为什么需要闭环评测

分段质量不能只靠人工观察。一个看起来完整的 chunk，可能在向量检索中表现不好；一个看起来很短的 chunk，可能因为缺少标题上下文而召回失败。
因此，本课题要使用下游检索指标验证分段策略。

12.2 对比策略

至少实现四种策略：

策略	说明
fixed_length	固定长度切分
heading_based	标题优先切分
semantic_based	语义相似度切分
structure_semantic_hybrid	本课题融合策略

12.3 评测数据集

建议准备 3 到 5 类文档：

- 1. 技术报告；
- 2. 课程任务书；
- 3. 法规制度；
- 4. 产品说明书；
- 5. 学术论文或百科长文。

每类准备 2 到 5 篇即可。

12.4 问题集构建

为每篇文档设计 10 到 20 个问题。

问题类型：

- 定义类；
- 参数查询类；
- 步骤查询类；
- 表格查询类；
- 原因分析类；
- 跨段落综合类；
- 代码说明类；
- 指标解释类。

每个问题需要标注：

```
1 {  
2   "question_id": "q_001",  
3   "question": "本智能体的分段质量如何评估？",  
4   "answer": "通过 Recall@k、nDCG 等下游检索指标评估。",  
5   "relevant_source_nodes": ["n_316", "n_323"],  
6   "relevant_keywords": ["Recall@k", "nDCG", "闭环评估"]  
7 }
```

12.5 向量检索流程

评测流程：

```
1 不同策略生成 chunk  
2   ↓  
3 对 chunk 生成 embedding  
4   ↓  
5 写入 FAISS / Milvus  
6   ↓  
7 对问题生成 embedding  
8   ↓  
9 检索 Top-k  
10  ↓  
11 判断是否命中相关 chunk  
12  ↓  
13 计算 Recall@k、nDCG、MRR
```

12.6 Recall@k

Recall@k 衡量前 k 个结果中是否召回相关片段。

```
1 Recall@k = |Relevant ∩ Retrieved_k| / |Relevant|
```

如果一个问题有 2 个相关 chunk，Top-5 召回了其中 1 个：

```
1 Recall@5 = 1 / 2 = 0.5
```

12.7 nDCG@k

nDCG 衡量检索排序质量。相关结果越靠前，得分越高。

```
1 DCG@k =  $\sum \text{rel}_i / \log_2(i + 1)$ 
2 nDCG@k = DCG@k / IDCG@k
```

12.8 MRR

MRR 衡量第一个相关结果出现的位置。

```
1 MRR = 平均值(1 / rank_of_first_relevant_result)
```

如果第一个相关结果在第 2 位，则该问题 MRR 为 0.5。

12.9 边界质量指标

除检索指标外，还要统计分段自身质量：

指标	计算方式
不破句率	未破句 chunk 数 / 总 chunk 数
特殊块保护率	完整保留的特殊块数 / 特殊块总数
目标长度命中率	token 数在目标区间的 chunk 数 / 总 chunk 数
过长率	超过 max_tokens 的 chunk 数 / 总 chunk 数
过短率	低于 min_tokens 的 chunk 数 / 总 chunk 数
回链完整率	source_refs 完整 chunk 数 / 总 chunk 数

12.10 内容组织质量指标

指标	评估方法
标签准确率	人工抽样检查
摘要忠实度	人工抽样或 LLM 辅助评估
实体标签准确率	对照实体表
标题路径正确率	人工检查
原文回链完整率	自动检查

12.11 评测报告内容

最终评测报告应包含：

1. 数据集说明；
 2. 策略配置；
 3. chunk 数量对比；
 4. 平均长度对比；
 5. 目标长度命中率；
 6. 特殊块保护率；
 7. Recall@k 对比；
 8. nDCG@k 对比；
 9. MRR 对比；
 10. 成功案例；
 11. 失败案例；
 12. 改进建议。
-

13. 可选进阶：QA / 指令数据合成

13.1 进阶方向定位

任务书将 QA / 指令数据合成列为进阶方向。它不是基础必做项，但如果完成，会增强项目亮点。

目标：

- 基于 chunk 生成问答对；
- 基于 chunk 生成指令数据；
- 标注可答性；
- 检查忠实度；
- 为微调或评测提供数据。

13.2 QA 生成流程

```
1 chunk 输入
2   ↓
3 判断 chunk 是否适合生成问题
4   ↓
5 生成候选问题
6   ↓
7 基于 chunk 生成答案
8   ↓
9 检查答案是否完全来自 chunk
10  ↓
11 标注可答性和忠实度
12  ↓
13 输出 QA 数据
```

13.3 QA 数据结构

```
1 {
2   "qa_id": "qa_001",
3   "chunk_id": "doc_001_chunk_0003",
4   "question": "智能分段需要满足哪些边界质量要求？",
5   "answer": "需要保证不破句、表格/公式/代码整体成块，并使目标长度区间命中率达到要求。",
6   "answerable": true,
7   "faithfulness_score": 0.96,
8   "evidence": ["不破句率 100%", "整体成块率 ≥ 95%"],
9   "source_refs": []
10 }
```

13.4 忠实度校验

忠实度校验可以采用：

1. 字符串证据匹配；
2. 关键词覆盖；
3. LLM 判断；
4. 人工抽样。

要求：

```
1 生成问答对可答性 ≥ 90%
2 忠实度 ≥ 90%
```

13.5 进阶功能注意事项

QA 合成容易出现幻觉，因此必须：

- 保留证据片段；
 - 不允许答案超出 chunk；
 - 对低置信问题丢弃；
 - 对生成数据打上来源标记；
 - 不把模型推测当作事实。
-

14. 服务接口设计

14.1 健康检查

```
1 GET /health
```

响应：

```
1 {
2   "status": "ok",
3   "version": "1.0.0"
4 }
```

14.2 分段接口

```
1 POST /v1/segment
```

请求：

```
1 {
2   "doc_id": "doc_001",
3   "doc_title": "示例文档",
4   "nodes": [],
5   "config": {
6     "strategy": "structure_semantic_hybrid",
7     "target_tokens": 768
8   }
9 }
```

响应:

```
1 {
2   "job_id": "job_001",
3   "doc_id": "doc_001",
4   "chunks": [],
5   "statistics": {},
6   "warnings": []
7 }
```

14.3 内容组织接口

```
1 POST /v1/organize
```

用途:

- 对已有 chunk 生成摘要;
- 生成关键词;
- 写入实体标签;
- 更新管理标签。

14.4 评测接口

```
1 POST /v1/evaluate
```

请求:

```
1 {
2   "doc_id": "doc_001",
3   "strategies": ["fixed_length", "heading_based", "structure_semantic_hybrid"],
4   "questions": []
5 }
```

响应:

```
1 {
2   "metrics": {
3     "fixed_length": {
4       "recall@5": 0.62,
5       "ndcg@5": 0.55
6     },
7     "structure_semantic_hybrid": {
8       "recall@5": 0.74,
9       "ndcg@5": 0.68
10    }
11  }
12 }
```

14.5 策略列表接口

```
1 GET /v1/strategies
```

响应:

```
1 {
2   "strategies": [
3     "fixed_length",
4     "heading_based",
5     "semantic_based",
6     "structure_semantic_hybrid"
7   ]
8 }
```


14.6 错误码设计

错误码	说明
INVALID_SCHEMA	输入 schema 不合法
EMPTY_DOCUMENT	文档内容为空
MISSING_NODE_ID	节点缺少 ID
TOKEN_LIMIT_EXCEEDED	超过硬长度限制
SUMMARY_FAILED	摘要生成失败
EVALUATION_FAILED	评测失败

15. 数据库与文件组织设计

15.1 项目目录建议

```
1 rag_chunk_agent/
2   app/
3     main.py
4     api/
5       segment.py
6       evaluate.py
7       health.py
8   core/
9     document_node.py
10    segmenter.py
11    boundary_scorer.py
12    length_controller.py
13    special_block.py
14    organizer.py
15    backlink.py
16  evaluation/
17    baselines.py
18    retriever.py
19    metrics.py
20    report.py
21  models/
22    schemas.py
23    configs.py
24  storage/
25    repository.py
26  data/
27    samples/
```

```
28     eval_questions/
29     outputs/
30     tests/
31     docs/
32     README.md
33     requirements.txt
34     docker-compose.yml
```

15.2 存储内容

建议存储：

- 输入文档快照；
- 分段配置；
- chunk 输出；
- 评测问题；
- 检索结果；
- 指标结果；
- 处理日志；
- 错误信息。

15.3 数据库表设计

可以使用 SQLite 或 PostgreSQL。

documents

字段	说明
doc_id	文档 ID
title	标题
source_file	来源文件
created_at	创建时间
metadata_json	元数据

chunks

字段	说明
chunk_id	chunk ID
doc_id	文档 ID
content	正文
summary	摘要
token_count	token 数
title_path_json	标题路径
source_refs_json	来源引用
strategy_info_json	策略信息

evaluation_results

字段	说明
eval_id	评测 ID
strategy	策略
recall_at_k	Recall@k
ndcg_at_k	nDCG@k
mrr	MRR
created_at	创建时间

16. 推荐技术栈

16.1 后端

技术	用途
Python 3.11	主开发语言
FastAPI	HTTP 服务

技术	用途
Pydantic	输入输出 schema 校验
Uvicorn	服务运行
SQLite / PostgreSQL	元数据存储
FAISS / Milvus	向量检索评测

16.2 NLP 与模型

技术	用途
sentence-transformers	语义向量
BGE / text2vec	中文 embedding
jieba / pkuseg	中文分词可选
KeyBERT / TextRank	关键词抽取可选
本地大模型 / API	摘要与标签生成可选

16.3 前端可选

技术	用途
Vue 3	演示页面
Element Plus	UI 组件
ECharts	指标图表

16.4 工程工具

技术	用途
Docker	部署
pytest	测试
Ruff / Black	代码规范
Markdown	报告与说明

16.5 选型原则

1. 能用规则解决的不用大模型；
 2. 大模型用于摘要、标签、疑难语义判断；
 3. embedding 用于语义边界和检索评测；
 4. 服务接口必须稳定；
 5. 评测脚本必须可重复运行。
-

17. 开发阶段规划

17.1 第一阶段：需求与数据结构设计

目标：

- 明确输入输出；
- 设计 DocumentNode；
- 设计 Chunk schema；
- 准备样例数据。

任务：

1. 整理任务书要求；
2. 确定能力边界；
3. 定义输入 JSON；
4. 定义输出 JSON；
5. 准备 2 到 3 篇测试文档；
6. 写接口草案。

交付：

- 数据结构说明；
- 示例输入；
- 示例输出；
- 接口草案。

17.2 第二阶段：基础分段实现

目标：

- 实现标题优先分段；
- 实现长度控制；

- 实现 `source_refs` 回链。

任务：

1. 构建标题树；
2. 给节点补充 `title_path`；
3. 实现候选块生成；
4. 实现 `token` 统计；
5. 实现过长拆分；
6. 实现过短合并；
7. 输出 `chunk JSON`。

交付：

- `POST /v1/segment`；
- 基础分段结果；
- 分段统计。

17.3 第三阶段：结构/语义融合增强

目标：

- 加入 `embedding` 语义边界；
- 加入特殊块保护；
- 加入 `overlap`。

任务：

1. 接入 `embedding` 模型；
2. 计算相邻段落相似度；
3. 实现边界评分；
4. 保护表格、公式、代码；
5. 实现表格长块处理；
6. 实现上下文 `overlap`。

交付：

- 融合分段策略；
- 特殊块保护示例；
- 边界调试信息。

17.4 第四阶段：内容组织实现

目标：

- 生成摘要；
- 生成关键词；
- 生成实体标签；
- 生成管理标签。

任务：

1. 实现规则关键词抽取；
2. 实现主题标签映射；
3. 接入摘要生成；
4. 实体标签写入；
5. 摘要忠实度抽样检查；
6. 输出组织后的 `chunk`。

交付：

- `chunk` 摘要；
- 关键词；
- 实体标签；
- 管理标签。

17.5 第五阶段：评测闭环实现

目标：

- 与固定长度切分比较；
- 输出 `Recall@k`、`nDCG`、`MRR`。

任务：

1. 实现 `fixed_length` baseline；
2. 实现 `heading_based` baseline；
3. 构建问题集；
4. 标注相关片段；
5. 建立向量索引；
6. 执行检索；
7. 计算指标；
8. 输出报告。

交付：

- 评测脚本；

- 指标对比表；
- 实验报告。

17.6 第六阶段：演示与文档完善

目标：

- 支持最终展示和验收。

任务：

1. 整理 README；
2. 整理接口文档；
3. 准备演示数据；
4. 准备运行脚本；
5. 准备前端页面或可视化报告；
6. 整理失败案例分析。

交付：

- 可运行服务；
 - 示例数据；
 - 演示说明；
 - 技术报告。
-

18. 小组分工

18.1 组长

负责：

- 总体架构；
- 需求拆解；
- 分段策略设计；
- 模块协调；
- 报告整合；
- 最终验收。

重点任务：

- 设计结构/语义融合分段算法；
- 设计边界评分机制；

- 组织评测实验。

18.2 解析与接口负责人

负责：

- 输入适配；
- DocumentNode 结构；
- FastAPI 接口；
- source_refs 回链；
- 示例输入输出。

重点任务：

- 保证智能体可被上游调用；
- 保证输入输出 schema 稳定。

18.3 内容组织负责人

负责：

- 摘要生成；
- 关键词抽取；
- 主题标签；
- 实体标签；
- 管理标签。

重点任务：

- 控制摘要忠实度；
- 设计标签体系；
- 输出组织化 chunk。

18.4 评测与前端负责人

负责：

- baseline 实现；
- 向量检索；
- 指标计算；
- 评测报告；
- 可视化页面。

重点任务：

- 证明智能分段相对固定长度切分有提升；

- 准备演示案例。

19. 验收指标对照

19.1 功能验收

功能	验收方式
输入结构化全文	使用示例 JSON 调用接口
输出 chunk 序列	检查 chunks 字段
标题路径	检查 title_path
长度控制	检查 token_count
表格保护	检查 table chunk
摘要生成	检查 summary
标签生成	检查 keywords / topic_tags
原文回链	检查 source_refs
评测指标	检查 evaluation report

19.2 指标验收

指标	目标
不破句率	100%
表格/公式/代码整体成块率	≥ 95%
目标长度区间命中率	≥ 90%
Recall@k / nDCG 提升	相对固定长度基线 ≥ 10% 或给出显著提升证据
语义完整性	≥ 85%
内容标签准确率	≥ 85%
摘要忠实度	≥ 90%
原文回链完整率	100%
QA 可答性	进阶 ≥ 90%
QA 忠实度	进阶 ≥ 90%

19.3 工程验收

工程项	要求
服务启动	一条命令可启动
接口文档	有请求响应示例
示例数据	有输入与输出样例
评测脚本	可复现指标
日志	可查看处理过程
配置	支持策略参数调整
部署	可本地或 Docker 运行

20. 与开源/通用方案的差异

20.1 与固定长度切分器的差异

固定长度切分器只关注长度，本智能体关注：

- 标题结构；
- 语义完整；
- 特殊块保护；
- 标签摘要；
- 来源回链；
- 检索指标。

20.2 与通用 RAG 框架的差异

LangChain、LlamaIndex、Haystack 等框架提供 splitter、retriever、vector store 等组件，但通常不负责：

- 复杂文档结构树；
- 表格/公式/代码保护；
- 分段质量闭环评估；
- chunk 内容组织；
- 原文锚点级回链；
- 治理流水线可审计。

本智能体可借鉴这些框架，但目标是做深“分段与内容组织”这一专项能力。

20.3 与文档解析工具的差异

Docling、Unstructured 等工具更偏向将原始文档解析成结构化元素。

本智能体的重点是：

1 | 解析之后，如何把结构化内容组织成高质量 RAG chunk

两者关系是上下游，不是替代关系。

20.4 与完整 RAG 平台的差异

RAGFlow 这类平台覆盖完整知识库和问答流程。本智能体不做完整平台，而是作为可插拔能力：

1	输入结构化全文
2	输出高质量 chunk

这使项目边界更清晰，也更适合课程实现。

21. 关键攻关点

21.1 攻关点一：结构与语义融合

难点：

- 标题边界和语义边界可能不一致；
- 长度控制可能破坏语义完整；
- 表格和代码可能导致超长。

解决思路：

- 标题边界作为硬优先级；
- 语义相似度作为辅助边界；
- 长度作为工程约束；
- 特殊块设置保护规则。

21.2 攻关点二：特殊块保护

难点：

- 表格跨页；
- 表格太长；
- 代码块过长；
- 公式需要解释文字；
- 列表不能断裂。

解决思路：

- 特殊块整体成块；
- 长表格按行组切分并重复表头；

- 代码按函数拆分；
- 公式与说明合并；
- 列表按一级项切分。

21.3 攻关点三：长度与 overlap 平衡

难点：

- chunk 太短召回困难；
- chunk 太长噪声多；
- overlap 太多增加冗余；
- overlap 太少缺上下文。

解决思路：

- 使用目标长度区间；
- 对不同类型 chunk 使用不同 overlap；
- 标题路径作为轻量上下文；
- 特殊块使用上下文补全。

21.4 攻关点四：内容组织可信度

难点：

- 摘要可能幻觉；
- 标签可能过泛；
- 实体可能不标准；
- 元数据可能丢失。

解决思路：

- 摘要只基于原文；
- 标签优先规则生成；
- 实体优先使用上游标准实体；
- 所有生成结果标记来源和置信度。

21.5 攻关点五：评测闭环

难点：

- 分段好坏难以主观判断；
- 问题集构建需要人工；
- 检索结果受 embedding 模型影响；
- 指标需要可复现。

解决思路：

- 设计固定数据集；
 - 固定 embedding 模型；
 - 固定检索参数；
 - 多策略对比；
 - 保留实验配置。
-

22. 风险与应对

22.1 文档结构不完整

风险：

上游输入没有完整标题、页码或节点类型。

应对：

- 设置降级策略；
- 用规则识别标题；
- 缺页码时保留 node_id；
- warnings 中记录问题。

22.2 语义模型效果不稳定

风险：

embedding 模型对专业术语理解不足。

应对：

- 提高结构边界权重；
- 使用行业关键词辅助；
- 可替换 embedding 模型；
- 对比不同模型效果。

22.3 表格过长

风险：

表格整体成块会超过长度上限。

应对：

- 行组拆分；
- 重复表头；

- 父子 chunk;
- 标记 oversized_table。

22.4 摘要生成幻觉

风险:

LLM 生成摘要加入原文没有的信息。

应对:

- 使用严格提示词;
- 限制摘要长度;
- 抽样人工检查;
- 低置信摘要标记;
- 必要时关闭摘要生成。

22.5 评测数据不足

风险:

问题集太少, 指标不稳定。

应对:

- 每篇文档至少 10 个问题;
- 覆盖多种问题类型;
- 人工标注相关片段;
- 保留失败案例分析。

22.6 项目范围过大

风险:

同时做解析、分段、摘要、评测、前端, 时间不够。

应对:

- 优先实现分段核心;
 - 摘要和 QA 作为增强;
 - 前端可简化为报告或 JSON 可视化;
 - 保证评测闭环优先。
-

23. 最终交付物清单

23.1 必交付

1. 可运行的智能分段服务；
2. 标准 HTTP API；
3. 输入输出 schema；
4. 分段核心算法；
5. 内容组织模块；
6. 原文回链机制；
7. 示例输入数据；
8. 示例 chunk 输出；
9. 固定长度基线；
10. 检索评测脚本；
11. 指标对比结果；
12. README；
13. 接口文档；
14. 技术报告。

23.2 建议交付

1. Docker 部署文件；
2. 前端演示页面；
3. 分段结果可视化；
4. 失败案例分析；
5. 摘要忠实度抽样报告；
6. 标签准确率抽样报告。

23.3 进阶交付

1. QA 数据生成模块；
2. 指令数据生成模块；
3. QA 可答性评测；
4. QA 忠实度评测；
5. 父子 chunk 机制；
6. 多粒度分段策略；

7. 检索与训练两种策略切换。

24. 核心伪代码

24.1 主流程伪代码

```
1 def segment_document(request):
2     doc = validate_and_normalize(request)
3     tree = build_document_tree(doc.nodes)
4     mark_special_blocks(tree)
5
6     candidates = build_candidates_by_heading(tree)
7     chunks = []
8
9     for candidate in candidates:
10         if is_special_block(candidate):
11             chunks.extend(handle_special_block(candidate))
12             continue
13
14         if token_count(candidate) <= config.max_tokens:
15             chunks.append(candidate)
16         else:
17             sub_chunks = split_by_semantic_boundary(candidate, config)
18             chunks.extend(sub_chunks)
19
20     chunks = merge_short_chunks(chunks, config)
21     chunks = apply_overlap(chunks, config)
22     chunks = enrich_metadata(chunks, tree)
23     chunks = generate_organization_info(chunks, config)
24     stats = compute_statistics(chunks)
25
26     return {
27         "doc_id": doc.doc_id,
28         "chunks": chunks,
29         "statistics": stats
30     }
```

24.2 边界评分伪代码

```
1 def score_boundary(left, right, context):
2     score = 0
3
4     if right.starts_with_heading:
5         score += 3.0
```

```

6
7     if left.ends_with_sentence:
8         score += 2.0
9     else:
10        score -= 5.0
11
12    semantic_drop = 1 - cosine(left.embedding, right.embedding)
13    score += semantic_drop * 2.0
14
15    length_score = compute_length_score(context.current_tokens)
16    score += length_score
17
18    if boundary_breaks_table_or_code(left, right):
19        score -= 10.0
20
21    return score

```

24.3 过短合并伪代码

```

1  def merge_short_chunks(chunks, config):
2      result = []
3      i = 0
4
5      while i < len(chunks):
6          current = chunks[i]
7
8          if current.token_count >= config.min_tokens:
9              result.append(current)
10             i += 1
11             continue
12
13         if i + 1 < len(chunks):
14             next_chunk = chunks[i + 1]
15             if can_merge(current, next_chunk, config):
16                 merged = merge(current, next_chunk)
17                 merged.strategy_info["merge_reason"] =
"below_min_tokens_same_title_path"
18                 result.append(merged)
19                 i += 2
20                 continue
21
22             result.append(current)
23             i += 1
24
25     return result

```

24.4 特殊块处理伪代码

```
1 def handle_special_block(block):
2     if block.type == "table":
3         if token_count(block) <= config.hard_max_tokens:
4             return [make_table_chunk(block)]
5         else:
6             return split_long_table_with_header(block)
7
8     if block.type == "code":
9         if has_functions(block):
10            return split_code_by_function(block)
11            return [make_code_chunk(block)]
12
13     if block.type == "formula":
14         return [merge_formula_with_explanation(block)]
15
16     return [block]
```

24.5 评测伪代码

```
1 def evaluate_strategies(doc, questions, strategies):
2     results = {}
3
4     for strategy in strategies:
5         chunks = segment_with_strategy(doc, strategy)
6         index = build_vector_index(chunks)
7
8         retrieved = {}
9         for q in questions:
10            topk = index.search(q.question, k=5)
11            retrieved[q.question_id] = topk
12
13         metrics = compute_retrieval_metrics(
14             questions=questions,
15             retrieved=retrieved
16         )
17
18         results[strategy] = metrics
19
20     return results
```